

Diffusion Drafts, AR Verifies: Accelerating Document OCR with Self-Speculative Decoding

Dohyun Kim^{1,2}, Sungjun Han¹, Hyungguk Kim¹, Yusik Kim¹, Jamin Shin¹,
Paul Hongsuck Seo², Hongjoon Ahn^{1,3}

¹Trillion Labs, ²Korea University, ³Seoul National University

Abstract

Autoregressive OCR vision–language models accurately convert document images into text and structured markup, but require one sequential decoding step per output token, limiting inference speed. Unlike open-ended text generation, OCR outputs are strongly grounded in the input image, making diffusion-based parallel generation promising. However, when several tokens are predicted in one diffusion step, each is predicted before the others are known. Committing them directly can therefore introduce errors. We therefore introduce GRAVITYOCR, a parameter-shared AR–block-diffusion model jointly trained for parallel drafting and causal AR verification. Verifying drafts before commitment lets the model commit multiple output tokens per round without a separate drafting network. The causal AR path also enables GRPO with sequence- and structure-level OCR rewards, avoiding diffusion-trajectory likelihood estimation while updating the shared drafter parameters. On OmniDocBench v1.6, AR-path GRPO improves the Overall score from 94.92 to 95.16 without reducing diffusion drafting efficiency, while the final model remains close to the original GLM-OCR score of 95.48. In an SGLang serving deployment, GRAVITYOCR commits an average of 9.7 output tokens per forward pass and achieves a **3.94× decode-only speedup** on region crops and a **1.32× end-to-end page-processing speedup** over AR decoding.

 [Code & Documentation](#)  [Model Weights](#)

1. Introduction

Generative OCR vision–language models have emerged as a leading approach to document understanding, converting document images into serialized sequences of plain text and structured markup such as HTML and LaTeX. Most of these models generate their outputs autoregressively, requiring one sequential decoding step for each output token. This sequential dependency directly affects latency and throughput when OCR models are deployed at scale. Recent OCR systems have therefore explored faster generation through multi-token prediction and speculative decoding (Chen et al., 2023; Duan et al., 2026; Leviathan et al., 2023; Li et al., 2026). In parallel, diffusion-based OCR models have begun to generate multiple output tokens simultaneously (Dong et al., 2026; Man et al., 2026).

OCR is particularly amenable to parallel generation because many output positions are directly constrained by the input image, unlike in open-ended text generation (Dong et al., 2026; Man et al., 2026). Masked diffusion provides a natural mechanism for exploiting this property: multiple output positions can be recovered together rather than generated strictly one at a

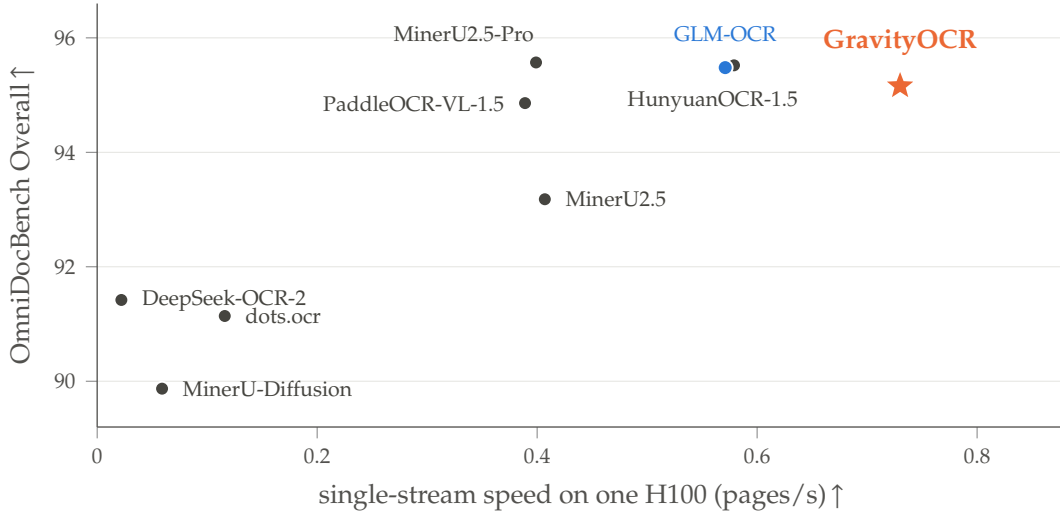


Figure 1 | **Accuracy–speed landscape of document OCR systems.** OmniDocBench v1.6 Overall vs. single-stream page rate, with the same evaluation set used across systems for each metric. GravityOCR is the fastest system measured while maintaining top-tier accuracy.

time (Austin et al., 2021; Sahoo et al., 2024). This makes diffusion-based decoding a promising direction for reducing the inference cost of OCR.

Visual grounding, however, does not eliminate sequential dependencies among OCR output tokens. Text must preserve its reading order, while tags and delimiters in tables and formulas must remain structurally consistent. Under causal AR decoding, each token is predicted after the preceding tokens have become available. In a parallel diffusion step, by contrast, positions predicted together cannot condition on the tokens simultaneously selected at the other positions. Directly committing these predictions can therefore produce tokens that are individually plausible under the partially resolved block but inconsistent in the completed output. Figure 2 illustrates this failure mode under confidence-based parallel decoding (Wu et al., 2025b).

Motivated by this failure mode, we introduce GRAVITYOCR, a parameter-shared AR–diffusion model inspired by recent AR–diffusion hybrids and diffusion-based drafting methods (Chen et al., 2026; Fu et al., 2026; Liu et al., 2025b; Wu et al., 2026a). GRAVITYOCR uses block diffusion (Arriola et al., 2025) to propose an entire token block in one forward pass and causal AR decoding to verify the proposal before commitment. The causal path commits only the longest draft prefix consistent with its token-by-token predictions. With top-1 verification, the resulting self-speculative decoder reproduces the causal AR greedy output in exact arithmetic (Appendix B.3 measures the agreement under bf16 serving kernels). Successful drafts allow decoding to advance by multiple tokens, whereas a draft–verifier disagreement limits the accepted draft length and attainable acceleration. Because both prediction paths share the same parameters, the method requires neither a separate drafting network nor an auxiliary prediction head.

We instantiate GRAVITYOCR by adapting GLM-OCR (Duan et al., 2026). The causal AR verifier determines the final output, so we apply GRPO through this path using sequence- and structure-level OCR rewards. This updates the shared parameters without estimating likelihoods over diffusion trajectories. On OmniDocBench v1.6, our final model retains 99.7% of the original GLM-OCR model’s Overall score (95.16 vs. 95.48), while self-speculative decoding substantially reduces the number of sequential decoding steps (Figure 1).

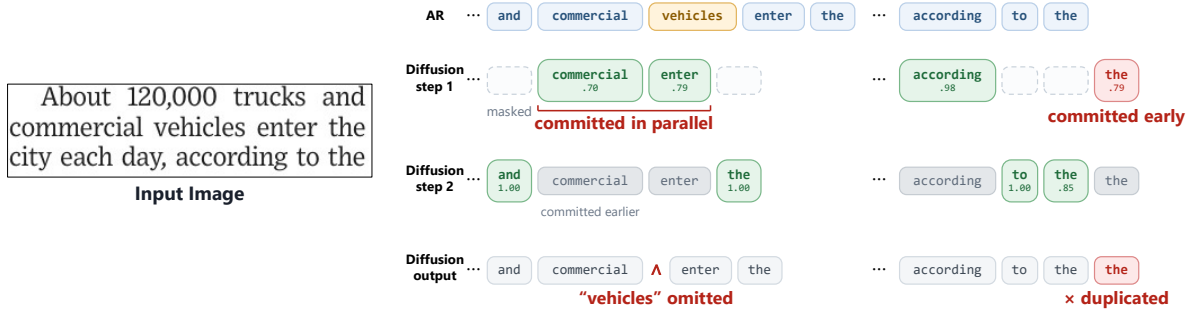


Figure 2 | **Failure case of confidence-based parallel commitment.** At $\tau_c=0.7$, commercial and enter are committed together while the intervening position remains unresolved, ultimately omitting vehicles. Similarly, the final the is committed before the two positions to its left are resolved, producing the repeated span to the the. The causal AR output from the same checkpoint is shown for comparison.

Our contributions are summarized as follows:

- We show that a pretrained causal AR OCR model can be adapted into a parameter-shared block-diffusion drafter and causal AR verifier, enabling faster decoding with minimal accuracy degradation and without requiring a separate drafting network.
- We implement full-backbone self-speculative decoding in SGLang (Zheng et al., 2024), achieving 9.7 tokens per forward (TPF), where both diffusion drafting and causal verification forwards are counted. This yields a 3.94 $\times$ speedup in token generation and a 1.32 $\times$ speedup in end-to-end page processing over causal AR decoding.
- We compare direct block-diffusion and self-speculative decoding on OCR, characterizing their accuracy-parallelism trade-off. We further show that GRPO applied only through the causal AR path improves the OmniDocBench Overall score from 94.92 to 95.16 while preserving the efficiency of the shared diffusion drafter.

2. Preliminaries

Task Formulation. Given an image I and a task prompt c , let $\mathbf{y} = (y_1, \dots, y_N)$ denote the corresponding target token sequence, which may encode plain text, an HTML table, or a LaTeX formula. Let p_θ^{AR} denote the conditional distribution induced by the decoder under causal attention. The autoregressive likelihood of $\mathbf{y}$ is factorized as

$$p_\theta^{\text{AR}}(\mathbf{y} \mid I, c) = \prod_{i=1}^N p_\theta^{\text{AR}}(y_i \mid I, c, \mathbf{y}_{<i}), \quad (1)$$

with the negative log-likelihood objective

$$\mathcal{L}_{\text{AR}} = - \sum_{i=1}^N \log p_\theta^{\text{AR}}(y_i \mid I, c, \mathbf{y}_{<i}). \quad (2)$$

At inference, each token is generated only after its preceding tokens become available, so an output of length N requires N sequential decoding steps.

Block Diffusion. Rather than generating one token at a time, masked diffusion language models iteratively reconstruct multiple masked positions at each denoising step (Austin et al., 2021; Nie et al., 2025; Ye et al., 2025). Block diffusion partitions the output sequence $\mathbf{y}$ into contiguous blocks $\mathbf{y}^{(b)}$, each containing at most B tokens (Arriola et al., 2025; Wu et al., 2025a). Here, b indexes the blocks in left-to-right order. Generation is causal across blocks: the current block $\mathbf{y}^{(b)}$ conditions on the completed prefix $\mathbf{y}^{(<b)} = [\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(b-1)}]$. Within the current block, attention is bidirectional. Let p_θ^{diff} denote the conditional distribution induced by the decoder under this block-causal attention pattern. For a sampled noise level t , let $\mathbf{y}_t^{(b)}$ denote the corrupted block obtained by replacing the positions in $\mathcal{M}_t^{(b)}$ with a dedicated mask token, denoted by $[M]$. The corresponding denoising objective is

$$\mathcal{L}_{\text{diff}} = -\mathbb{E}_{b,t,\mathcal{M}_t^{(b)}} \left[w(t) \sum_{j \in \mathcal{M}_t^{(b)}} \log p_\theta^{\text{diff}} \left(y_j^{(b)} \mid I, c, \mathbf{y}^{(<b)}, \mathbf{y}_t^{(b)} \right) \right], \quad (3)$$

where j indexes token positions within block b , and $w(t)$ denotes the timestep-dependent loss weight.

At inference, the current block is initialized with B mask tokens. At each denoising step, the model predicts all remaining masked positions in parallel, after which a selection rule—such as random selection, confidence-based top- k selection, or confidence thresholding—determines which predictions are unmasked and retained (Nie et al., 2025; Sahoo et al., 2024; Wu et al., 2025b). The updated block is then used as input for the next denoising step. Once the block is complete, it is appended to the prefix. Because completed prefix blocks remain unchanged, their KV states can be cached and reused when decoding subsequent blocks (Arriola et al., 2025; Wu et al., 2025b).

Speculative Decoding. Speculative decoding accelerates AR generation by allowing a drafter to propose multiple future tokens, which the target AR model evaluates in parallel (Chen et al., 2023; Leviathan et al., 2023). Under greedy decoding, consecutive draft tokens are accepted while they match the target model’s predictions. Under stochastic decoding, the standard acceptance–rejection rule preserves the target distribution. Our shared-model realization is described in Section 3.2.

3. Method

Our goal is to reduce the decoding cost of OCR while retaining the recognition accuracy of causal AR decoding. Motivated by the failure mode analyzed in Figure 2, we separate parallel drafting from token commitment: a block-diffusion path proposes multiple tokens in parallel, while a causal AR path verifies the proposals before they are committed. Sections 3.1 to 3.3 describe joint AR–diffusion adaptation, self-speculative decoding, and reinforcement learning through the causal AR path with sequence- and structure-level OCR rewards, respectively.

3.1. Learning a Shared Block-Diffusion Drafter and AR Verifier

AR-to-Diffusion Conversion. We adapt a pretrained AR OCR model into a parameter-shared model that supports both causal AR and block-diffusion prediction. The two modes share the vision encoder, language decoder, and LM head; the only architectural addition is a learned embedding for the dedicated mask token, denoted by $[M]$ in our figures. We introduce neither a separate drafter network nor an auxiliary prediction head.

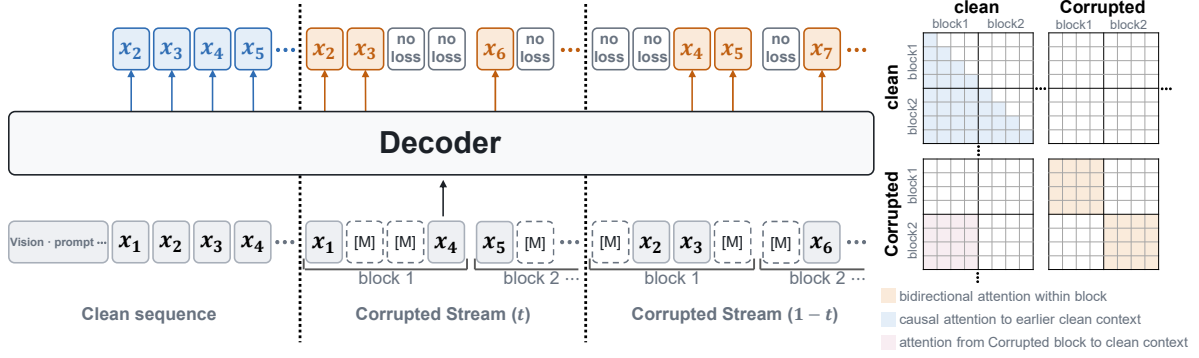


Figure 3 | **Joint AR-diffusion training in a single forward pass.** The clean response stream provides causal next-token supervision, while two complementary corrupted streams with masking ratios t and $1 - t$ provide denoising supervision over disjoint masked targets. All streams share the visual and prompt context, decoder, and LM head. The right panel contrasts token-level causal attention in the clean stream with causal cross-block and bidirectional within-block attention in the corrupted streams.

Joint AR-Diffusion Training. We jointly train the shared model for self-speculative decoding. Under block diffusion, the tokens decoded in the same step are predicted from the same partially masked block and cannot condition on one another’s values, so committing them directly can leave the completed output inconsistent. We therefore also train the model on next-token prediction over the clean sequence, preserving the causal AR path that verifies each draft and rejects inconsistent proposals before they are committed. Figure 3 provides an overview of our training procedure. At each training step, the shared model processes one clean response stream alongside two complementary corrupted response streams in a single forward pass. The clean stream provides causal next-token supervision for the AR verifier, whereas the corrupted streams provide denoising supervision for the block-diffusion drafter. All three response streams are conditioned on the same image I and task prompt c , both of which remain uncorrupted.

For each block $\mathbf{y}^{(b)}$, we sample a noise level $t \sim \mathcal{U}(0, 1)$ and mask each response token in the block independently with probability t , giving a set of masked indices $\mathcal{M}_t^{(b)}$. The first corrupted stream replaces the tokens indexed by $\mathcal{M}_t^{(b)}$ with the mask token, whereas the second corrupted stream masks the complementary positions of the same block. The two streams therefore have expected masking ratios t and $1 - t$, respectively. Because the two masks are complementary, every response token, including the B end-of-sequence tokens appended to each response (Appendix A), is masked and receives denoising supervision in exactly one of the two streams, while image and prompt tokens are never masked or supervised. This complementary masking scheme follows Fast-dLLM v2 (Wu et al., 2025a).

The clean stream uses causal attention, so each target token y_i is predicted using only the image I , task prompt c , and preceding tokens $\mathbf{y}_{<i}$. Within each corrupted stream, tokens in block b can attend to all positions in the same corrupted block. They can also attend to the preceding clean response blocks $\mathbf{y}^{(<b)}$, but not to the current or future clean blocks. Both clean and corrupted streams predict y_i from the logit at position $i - 1$. In corrupted streams, denoising supervision applies only to masked targets.

Training Objective. The losses from the clean and corrupted streams are combined as

$$\mathcal{L} = \mathcal{L}_{\text{AR}} + \lambda \mathcal{L}_{\text{diff}}, \quad (4)$$

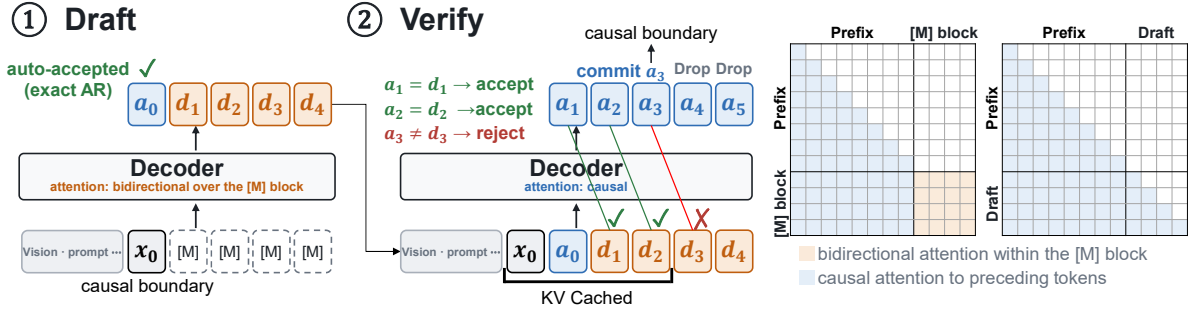


Figure 4 | **Self-speculative decoding.** Given a causal boundary token x_0 followed by B mask tokens, the shared decoder produces in a single forward pass one causal token a_0 that is accepted by construction and B parallel draft tokens $d_{1:B}$, attending bidirectionally within the mask block and causally to the cached prefix; the draft is passed on without iterative unmasking or confidence-based selection. A causal verification pass over $a_0, d_{1:B}$ produces $a_{1:B+1}$ and commits the longest AR-consistent draft prefix and the next AR token while simultaneously constructing the KV cache for the accepted prefix, so no additional cache-construction pass is needed; the states of rejected tokens are discarded. The final AR token becomes the causal boundary for the next round and obtains its KV state there.

where λ controls the relative weight of the diffusion objective. In implementation, we average the AR loss in Equation 2 over supervised clean-stream tokens and the diffusion loss in Equation 3 over masked targets pooled across both corrupted streams, using $w(t) = 1$. We normalize the combined loss by $1 + \lambda$ and use $\lambda = 1$, giving each objective weight 0.5, except in the no-AR-loss ablation in Table 5.

3.2. Self-Speculative Decoding

In direct block-diffusion decoding, token commitment is based on the diffusion model’s confidence. An incorrect token can therefore be committed before the block is fully resolved; once fixed, it may lead to omissions, repetitions, or structural errors. We avoid this failure mode by separating parallel drafting from token commitment. The block-diffusion path proposes a token block in parallel, while the causal AR path verifies the draft and determines which tokens enter the output. The verifier accepts only the longest draft prefix that matches its next-token predictions, preserving causal AR decoding while allowing successful drafts to advance generation by multiple tokens. Figure 4 illustrates the resulting procedure.

Each round starts from the last committed token x_0 , followed by B mask tokens. The boundary position x_0 uses causal attention, while the masked positions attend to the committed prefix and bidirectionally within the mask block. In a single forward pass, the shared decoder produces the next AR token a_0 from the boundary position and the draft tokens $d_{1:B}$ from the masked positions. Since a_0 is already a causal AR prediction, it is committed directly.

To verify the draft tokens, we feed the sequence $[a_0, d_{1:B}]$ into the shared decoder under token-level causal attention. This verification pass produces the AR predictions $a_{1:B+1}$. Following Section 2, we accept the longest draft prefix $d_{1:A}$ satisfying $d_j = a_j$ for every $1 \leq j \leq A$, and append the next AR token a_{A+1} . This final token serves as the boundary token for the next round.

3.3. RL on the AR Path

The joint training objective in Section 3.1 provides token-level supervision for both causal AR prediction and block-diffusion drafting. However, OCR quality is evaluated on complete outputs using sequence- and structure-level criteria that are not directly optimized by token-level likelihood training. We therefore further optimize the jointly trained model using task-aware OCR rewards.

For multi-step masked diffusion, evaluating the sequence likelihood requires marginalizing over unmasking trajectories, so existing RL methods approximate it or apply policy gradients along the denoising trajectory (Zhan, 2025; Zhao et al., 2025). In GRAVITYOCR, however, the causal AR verifier determines the output of self-speculative decoding. We can therefore optimize the AR path using its exact autoregressive likelihood and apply standard GRPO directly. We fine-tune the full model through this path, updating the diffusion drafter through shared parameters without a diffusion-specific objective.

Task-Aware OCR Rewards. For each sampled output, we select the reward according to its OCR task. For plain-text recognition, we use normalized edit similarity, $1 - \text{NED}$, where NED is the normalized edit distance. For HTML table recognition, we combine structure-only TEDS (Zhong et al., 2020) with a cell-content similarity and penalize malformed or degenerate markup. For formula recognition, we use the edit similarity of canonicalized LaTeX, scaled down when the output fails well-formedness checks, as a rendering-free proxy for CDM (Wang et al., 2024b). All rewards lie in $[0, 1]$; exact definitions and coefficients are given in Appendix E.

GRPO on the Causal AR Path. For each input, we sample a group of outputs from the jointly trained model’s causal AR path and score them with the corresponding task-aware reward. GRPO (Shao et al., 2024) computes relative advantages within each group and updates the model using causal token-level log-probabilities. Because RL updates through the causal AR path are applied to the shared parameters, they can also change the block-diffusion drafter and its agreement with the verifier. As shown in Section 4.5, GRPO improves OCR accuracy while tokens per forward remain essentially unchanged, indicating that drafter-verifier agreement is preserved after RL.

4. Experiments

4.1. Setup

Model and Training. We initialize GRAVITYOCR from the released GLM-OCR checkpoint (Duan et al., 2026) and jointly fine-tune its vision encoder and language decoder for causal AR and block-diffusion prediction. In the original pipeline, PP-DocLayout-V3 (Sun et al., 2025) identifies document regions, the region-level OCR model decodes each crop, and the resulting outputs are assembled by the merge and post-processing pipeline. We retain the layout detector and assembly pipeline unchanged and apply the AR-to-diffusion adaptation only to the region-level OCR model. Unless otherwise specified, we use a block size of $B = 32$ during both training and inference, and construct each self-speculative draft with a single block-diffusion forward pass. The joint objective weighs the two losses equally ($\lambda = 1.0$). We train for 40,000 steps, processing $\sim 26\text{B}$ forward tokens in total; this count includes vision tokens and the three response streams of each example. Further optimization and implementation details are provided in

Appendix A. Unless otherwise stated, results use the final GRPO checkpoint, obtained after 40,000 joint-training steps and 500 GRPO steps (Section 4.5).

Training Data. The training pool contains 12.3M region-level examples assembled from predominantly public data, including DocGenome (Xia et al., 2024), Docmatix (Laurençon et al., 2024), PubTables-1M (Smock et al., 2022), FinTabNet (Zheng et al., 2021), SynthTabNet (Nassar et al., 2022), PubTabNet (Zhong et al., 2020), RVL-CDIP (Harley et al., 2015), DocLayNet (Pfitzmann et al., 2022), and the training split of UniMER (Wang et al., 2024a). Each example is a layout region cropped from a full page at native resolution, paired with a text target. For full-page sources, regions are detected with the same PP-DocLayout-V3 detector used at inference; table- and formula-only sources are used as whole crops. Targets are transcriptions of each crop produced by the base GLM-OCR model, except for table crops from sources that ship cell-level annotations (about 39% of the table stream), for which we use the original annotations converted to the evaluation markup convention. The pool consists of three streams—page text (6.49M regions), tables (2.90M), and formulas (2.89M)—and is predominantly English. To realize a fixed 60/20/20 stream ratio by example count, the table and formula streams are each subsampled once with a fixed seed to 2.16M examples, giving a 10.8M training set that is globally shuffled. We additionally hold out 1,000 pages (4,614 region crops) as an internal validation set. The pool contains no OmniDocBench pages (checked by document identifier, text shingles, table cells, and formula strings); PubTabNet is used through its training split only.

Benchmarks and Metrics. We use OmniDocBench v1.6 as our primary benchmark, which contains 1,651 document pages and evaluates document parsing under its official protocol and aggregation (Ouyang et al., 2025; Wang et al., 2026). We report the official Overall score together with the corresponding text, table, formula, and reading-order metrics. We additionally evaluate structured recognition on the PubTabNet validation set, comprising 9,115 tables, and on UniMER-Test for mathematical expression recognition (Wang et al., 2024a; Zhong et al., 2020). For PubTabNet, we report both full TEDS and structure-only TEDS, while UniMER is evaluated using CDM (Wang et al., 2024b). All models are evaluated using the same scoring pipeline for each benchmark.

Baselines. For in-family comparisons, we compare the original GLM-OCR model with causal AR decoding and its built-in MTP branch (Duan et al., 2026) against GRAVITYOCR’s causal AR, direct block-diffusion, and self-speculative decoding. Direct block-diffusion decoding serves as a diagnostic baseline that iteratively commits confidence-selected diffusion predictions without causal verification. For cross-model evaluation, we report the reproduced results of MinerU2.5 (Niu et al., 2025), MinerU2.5-Pro (Wang et al., 2026), MinerU-Diffusion (Dong et al., 2026), PaddleOCR-VL-1.5 (Cui et al., 2026), dots.ocr (rednote hilab, 2025), DeepSeek-OCR-2 (Wei et al., 2026), and HunyuanOCR-1.5 (Li et al., 2026) using the released implementations.

Efficiency Measurement. We measure tokens per forward (TPF) over the complete decoding run as

$$\text{TPF} = \frac{\text{number of committed output tokens}}{\text{number of forward passes}}, \quad (5)$$

where each block-diffusion drafting and each causal verification are counted separately as one forward pass. TPF is measured with a single request in flight (batch size 1), so one forward pass is one model invocation on that request, and the per-request vision and prompt prefill forward

Table 1 | **Main results on OmniDocBench v1.6 and page-processing speed.** Text and Order measure normalized edit distance over character sequences and text-block reading order, respectively; TEDS is tree-edit-distance similarity for tables, and CDM is character detection matching for formulas. Overall averages the three recognition axes. Page speed includes each system’s layout stage. TPF denotes output tokens committed per model forward. All results are our measurements; speed protocols are in Appendix C. [†]These modes use extra draft parameters; TPF counts tokens accepted per base-model verification, excluding drafting computation.

Model	Decode	Quality					Speed		
		Text [↓]	TEDS [↑]	CDM [↑]	Order [↓]	Overall [↑]	TPF [↑]	tok/s [↑]	pages/s [↑]
<u>External parsers</u>									
MinerU2.5	AR	0.045	0.881	0.959	0.130	93.18	1.0	559	0.407
MinerU2.5-Pro	AR	0.037	0.935	0.969	0.124	95.57	1.0	545	0.399
MinerU-Diffusion	diff	0.073	0.853	0.916	0.154	89.87	5.2	79	0.059
PaddleOCR-VL-1.5	AR	0.042	0.919	0.969	0.129	94.86	1.0	741	0.389
dots.ocr	AR	0.048	0.865	0.917	0.140	91.14	1.0	149	0.116
DeepSeek-OCR-2	AR	0.049	0.859	0.933	0.144	91.42	1.0	24	0.022
HunyuanOCR-1.5	AR	0.036	0.952	0.949	0.125	95.52	1.0	390	0.313
HunyuanOCR-1.5	DFlash	0.036	0.952	0.949	0.125	95.52	9.9 [†]	720	0.579
GLM-OCR (base)	AR	0.040	0.934	0.970	0.141	95.48	1.0	807	0.571
GLM-OCR (base)	MTP	0.040	0.934	0.970	0.141	95.48	3.7 [†]	667	0.472
<u>Ours</u>									
GravityOCR	AR	0.040	0.928	0.967	0.141	95.16	1.0	794	0.554
GravityOCR	self-spec	0.040	0.928	0.967	0.141	95.16	9.7	1,047	0.730

is not counted. We also report the number of drafted tokens accepted per round, excluding the boundary prediction a_0 and the verifier’s own token, which every round commits regardless of the draft. Under this convention, standard AR decoding has a TPF of 1 by construction. We additionally report decoding throughput in tokens per second (tok/s) and document throughput in pages per second (pages/s). Unless stated otherwise, GRAVITYOCR and the GLM-OCR AR baseline are served with SGLang (Zheng et al., 2024) and timed through its client interface; the GLM-OCR MTP row uses its official vLLM path, and each external system runs on its own official inference stack (vLLM, PaddleX, or native Transformers; see Appendix C). The Hugging Face Transformers rows of Table 3 are an in-process eager implementation, included as an unoptimized reference.¹

4.2. Main Results

OmniDocBench. Table 1 compares GRAVITYOCR with the original GLM-OCR model and representative document parsing systems on OmniDocBench v1.6, reporting recognition quality and page-processing speed side by side. GRAVITYOCR achieves an Overall score of 95.16, compared with 95.48 for the original GLM-OCR, retaining its document parsing quality within 0.32 points. Relative to the base model, TEDS and CDM drop only slightly. Apart from the original GLM-OCR, the Overall score of GRAVITYOCR is surpassed only by MinerU2.5-Pro and HunyuanOCR-1.5 among the evaluated parsers.

¹Because our training corpus is predominantly English, efficiency measurements use the English subset of OmniDocBench unless otherwise stated; quality scores are computed on the full evaluation set.

Table 2 | **Table and formula recognition.** PubTabNet table recognition on the full validation set (9,115 tables) and UniMER formula recognition, all rows measured by us under one pipeline per benchmark, with each system’s own task prompt and the same output normalisation.

Model	PubTabNet TEDS [†]	TEDS-struct [†]	UniMER CDM [†]
MinerU2.5	0.854	0.909	0.927
MinerU2.5-Pro	0.866	0.913	0.956
MinerU-Diffusion	0.734	0.858	0.952
PaddleOCR-VL-1.5	0.828	0.900	0.956
dots.ocr	0.893	0.935	0.910
DeepSeek-OCR-2	0.849	0.896	0.863
HunyuanOCR-1.5	0.852	0.903	0.942
GLM-OCR (base, AR)	0.803	0.858	0.963
GravityOCR	0.871	0.916	0.962

Page Processing Speed. The speed columns of Table 1 are measured on a 100-page subset of OmniDocBench, using a single H100 with one page in flight. Each model is evaluated through its released page-processing pipeline, with layout analysis, region recognition, and output assembly included in the wall-clock measurement. With self-speculative decoding, GRAVITYOCR commits an average of 9.7 output tokens per model forward and achieves 1,047 tok/s and 0.730 pages/s, the highest processing rates in the table. Its page throughput exceeds that of MinerU2.5-Pro at 0.399 pages/s and HunyuanOCR-1.5 with DFlash at 0.579 pages/s. The causal AR mode of the same checkpoint reaches 794 tok/s and 0.554 pages/s, corresponding to a 1.32× page-processing speedup from self-speculative decoding. GRAVITYOCR also exceeds the built-in MTP mode of the original GLM-OCR at 0.472 pages/s. Notably, the MTP mode remains slower than the original model’s causal AR mode at 0.571 pages/s despite advancing 3.7 tokens per forward, whereas the parallelism of GRAVITYOCR translates into a wall-clock page-processing gain.

Table and Formula Recognition. Beyond OmniDocBench, Table 2 evaluates GRAVITYOCR on PubTabNet and UniMER, dedicated benchmarks for table and formula recognition, respectively. On PubTabNet, GRAVITYOCR achieves a TEDS score of 0.871 and a TEDS-struct score of 0.916. TEDS evaluates both table structure and cell content, whereas TEDS-struct isolates the recovered table structure. Compared with the original GLM-OCR, GRAVITYOCR improves TEDS from 0.803 to 0.871 and TEDS-struct from 0.858 to 0.916. On UniMER, GRAVITYOCR achieves a CDM score of 0.962, closely matching the original GLM-OCR score of 0.963 and exceeding all evaluated external systems.

Taken together, these results show that our framework largely preserves the recognition performance of the base model across the evaluated benchmarks while enabling block-diffusion drafting and causal AR verification.

4.3. Decoding Efficiency

Inference Backend and Measurement Scope. Table 3 compares AR and self-speculative decoding under two inference backends, Hugging Face Transformers (Wolf et al., 2020) and SGLang (Zheng et al., 2024), using the same checkpoint and region inputs. For each backend, we report decode-only throughput for token generation and end-to-end throughput including vision encoding and prompt prefill. With the Transformers implementation, self-speculation

Table 3 | **AR vs. self-speculative decoding.** Decode-only and end-to-end throughput under Transformers and SGLang.

Backend	Decode	tok/s [†]		speedup
		decode-only	end-to-end	
Transformers	AR	53	51	—
Transformers	self-spec	293	161	5.53× / 3.20×
SGLang	AR	777	486	—
SGLang	self-spec	3057	844	3.94× / 1.74×

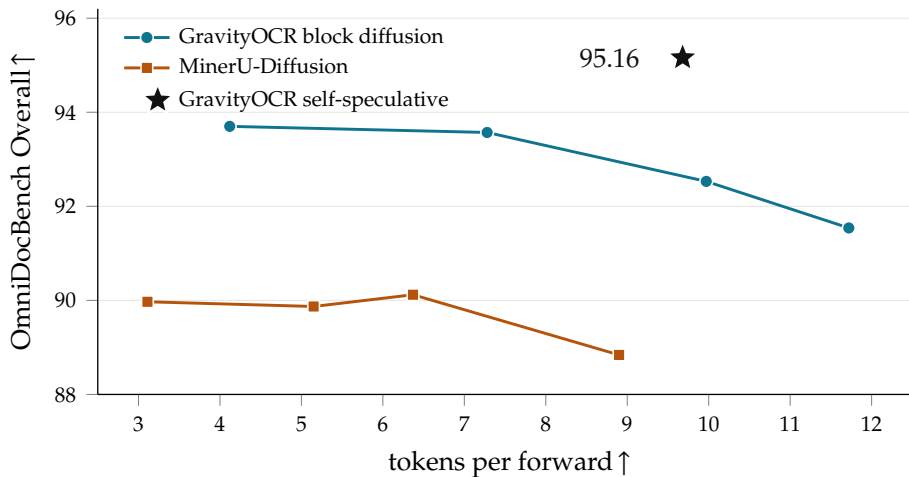


Figure 5 | **Quality and parallelism of diffusion decoding.** Curves sweep each system’s confidence threshold τ_c . Direct block-diffusion TPF counts forwards that commit tokens, excluding cache writes; self-speculative TPF counts both drafting and verification (Appendix C).

raises the decode-only rate from 53 to 293 tok/s (5.53×) and the end-to-end rate from 51 to 161 tok/s (3.20×). With SGLang, self-speculation raises decode-only throughput from 777 to 3,057 tok/s (3.94×) and end-to-end throughput from 486 to 844 tok/s (1.74×). The decode-only gain is larger in the Transformers implementation, whose eager execution is bound by kernel-launch overhead: a forward costs roughly the same whether it processes one token or a whole draft window, so the gain approaches the tokens-per-forward ratio. SGLang’s fused kernels make each forward cheaper but scale its cost with the number of tokens processed, and the fixed per-crop cost of vision encoding and prompt prefill becomes a larger share of a request whose decoding is much faster, which is why its end-to-end gain is smaller.

Direct Block-Diffusion vs. Self-Speculative Decoding. We compare direct block-diffusion and self-speculative decoding to measure the effect of causal AR verification. Direct block-diffusion commits confidence-selected predictions without verification, exposing the quality–parallelism trade-off of parallel commitment. Self-speculative decoding instead uses the diffusion path to propose a block and the causal AR path to determine which tokens are committed. Figure 5 compares this with MinerU-Diffusion, a representative diffusion-native OCR model. Across the evaluated confidence thresholds, increasing the number of tokens committed per forward is accompanied by lower recognition accuracy under direct diffusion decoding. The diffusion mode of GRAVITYOCR remains more accurate than MinerU-Diffusion throughout the sweep,

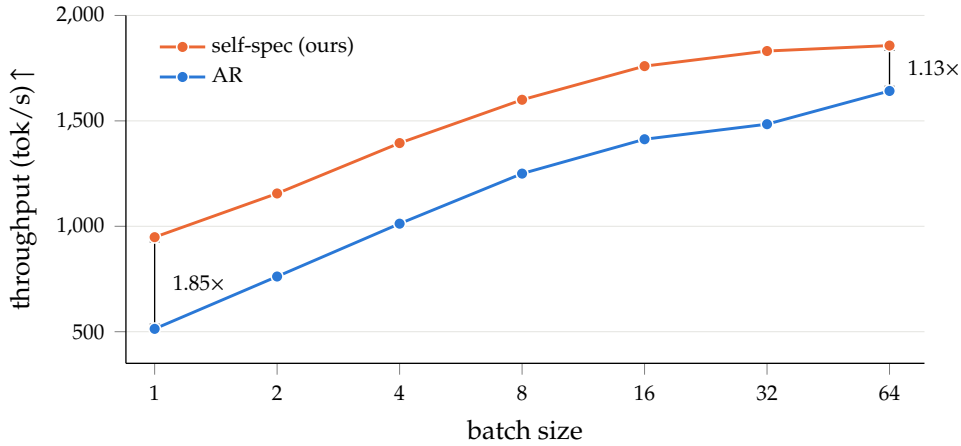


Figure 6 | **Throughput vs. batch size.** The speedup narrows with batch size: speculation leads by 1.85 $\times$ at batch 1 and by 1.13 $\times$ at batch 64. Protocol in Appendix C.

Table 4 | **Parallelism and speed by content type.** Accepted tokens is the number of draft tokens accepted per round. TPF divides committed output tokens by all draft and verification forwards pooled within each content type.

Type	crops	mean output		accepted tokens $\uparrow$	TPF $\uparrow$	tok/s $\uparrow$		speedup
		tokens	tokens			AR	self-spec	
Text	7,019	73	15.0	8.5	419	647	1.54 $\times$	
Formula	1,660	95	18.9	10.5	520	947	1.82 $\times$	
Table	243	872	25.0	13.5	732	2,462	3.36$\times$	
All	8,922	99	17.4	9.7	486	844	1.74 $\times$	

but exhibits the same trade-off. At comparable parallelism of approximately 10 tokens per forward, direct block-diffusion decoding achieves an Overall score of 92.53, whereas self-speculative decoding achieves 95.16 at 9.7 tokens per forward. This comparison shows that causal AR verification avoids the recognition loss associated with direct parallel commitment while retaining comparable parallelism.

Throughput Scaling with Batch Size. Batching can improve GPU utilization for causal AR decoding, potentially narrowing the relative advantage of self-speculation. Figure 6 evaluates SGLang throughput under increasing numbers of concurrently processed region inputs, including vision encoding, prompt prefill, and token generation. In this sweep, measured before RL, self-speculative decoding leads by 1.85 $\times$ at batch size one. As the batch size increases, batching exposes more parallel work from the AR decoder to the GPU, improving its utilization and narrowing the advantage of self-speculation. At batch size 64, AR decoding reaches 1,642 tok/s and self-speculative decoding reaches 1,857 tok/s. Although the advantage narrows at higher batch sizes, self-speculative decoding still achieves a 1.13 $\times$ higher throughput. These results show that self-speculative decoding provides its largest throughput gains in low-concurrency settings, where sequential AR decoding leaves more GPU capacity underutilized.

Decoding Efficiency by Output Type. Table 4 shows that table regions benefit more from self-speculative decoding than text regions. They accept 25.0 of the 32 drafted tokens per round,

Table 5 | **Effect of the auxiliary AR loss on self-speculative decoding.** Both models are 10k-step checkpoints that share the training setup except the AR loss weight.

	OmnidocBench Overall [†]	TPF [†]
no AR loss	93.64	6.96
AR loss	95.02	6.75

Table 6 | **GRPO on the AR path.** OmnidocBench v1.6 Overall and TPF for direct block-diffusion and self-speculative decoding before and after GRPO.

Decode	τ_c	Overall [†]			TPF [†]	
		before	after	Δ	before	after
Block diffusion	0.7	86.17	86.06	-0.11	14.85	15.43
Block diffusion	0.9	91.55	91.54	-0.01	11.89	11.72
Block diffusion	0.95	89.62	92.53	+2.91	10.20	9.97
Block diffusion	0.99	93.23	93.57	+0.34	7.18	7.28
Self-spec	—	94.92	95.16	+0.24	9.61	9.68

compared with 15.0 for text, and achieve speedups of 3.36 $\times$ and 1.54 $\times$, respectively. Table outputs are also longer, averaging 872 tokens compared with 73 for text, which reduces the relative cost of vision encoding and prompt prefill. Their higher draft acceptance may reflect the stronger syntactic constraints of structured outputs.

4.4. Effect of the AR Loss

Table 5 isolates the auxiliary AR loss. Even without this loss, the model can perform causal verification because it is initialized from a pretrained AR model. Removing AR supervision lowers the Overall score from 95.02 to 93.64, while TPF remains similar: 6.75 with the AR loss and 6.96 without it. The AR objective therefore preserves verifier accuracy with little change in drafting efficiency.

4.5. RL on the AR Path

GRPO on the AR Path. We evaluate whether AR-path GRPO improves OCR quality while preserving diffusion drafting efficiency. Table 6 compares direct diffusion and self-speculative decoding before and after GRPO.

Under self-speculative decoding, GRPO improves the OmnidocBench Overall score from 94.92 to 95.16, while TPF remains essentially unchanged at 9.61 and 9.68, respectively. Direct diffusion also retains similar or improved accuracy and similar TPF across the evaluated confidence thresholds, despite receiving no diffusion-specific training objective. These results show that AR-path GRPO improves OCR quality while preserving drafting efficiency.

5. Related Work

Document Parsing VLMs. Generative document parsing models translate document images into serialized text and structured markup. Early systems such as Nougat focused on image-to-markup transcription for scientific documents, while olmOCR extended generative parsing

to large-scale document conversion (Blecher et al., 2023; Poznanski et al., 2025). More recent OCR-specialized VLMs, including GLM-OCR, MinerU2.5-Pro, OvisOCR2, PaddleOCR-VL, dots.ocr, and DeepSeek-OCR, improve document parsing through compact architectures, specialized training data, and document-aware visual processing (Cui et al., 2025; Duan et al., 2026; Lu et al., 2026; rednote hilab, 2025; Wang et al., 2026; Wei et al., 2025). For example, DeepSeek-OCR compresses document images into compact optical contexts, PaddleOCR-VL adopts coarse-to-fine visual processing, and GLM-OCR combines layout analysis with region-level content generation. These studies primarily improve visual representation, training data, or the document-processing pipeline, while retaining causal AR generation for the output sequence.

A smaller body of work directly targets OCR decoding efficiency. GLM-OCR incorporates an MTP branch that predicts multiple future tokens from its AR backbone (Duan et al., 2026). Because this branch is included in our base OCR system, we use it as the direct in-model acceleration baseline. HunyuanOCR-1.5 instead adapts DFlash to OCR by training a separate block-diffusion drafter conditioned on features from the target AR model (Chen et al., 2026; Li et al., 2026). HSD accelerates document parsing at a different granularity: it constructs region-level drafts, verifies regions in parallel, and performs an additional page-level verification stage to preserve global document coherence (Liao et al., 2026). These approaches establish multi-token drafting and verification as practical directions for accelerating generative OCR, but differ in whether drafting is performed by auxiliary prediction heads, an external draft model, or a hierarchical document pipeline.

Diffusion Language Models. Discrete diffusion modeling for text originated from denoising objectives over corrupted token sequences (Austin et al., 2021; Sahoo et al., 2024), and was subsequently scaled to large diffusion language models such as LLaDA and Dream (Nie et al., 2025; Ye et al., 2025). Later work has focused on making diffusion generation compatible with efficient language-model serving. Block Diffusion introduces a block-wise formulation that supports reuse of completed context (Arriola et al., 2025). Fast-dLLM develops caching-aware inference techniques for pretrained diffusion language models (Wu et al., 2025b), while Fast-dLLM v2 proposes a data-efficient AR-to-block-diffusion adaptation scheme using complementary clean and corrupted training streams (Wu et al., 2025a). WeDLM follows a different approach, using causal attention and topological token reordering to enable parallel prediction while retaining compatibility with standard prefix KV caching (Liu et al., 2025a).

Diffusion decoding has recently been introduced to document OCR. DODO develops a block-discrete diffusion OCR model designed to mitigate the structural instability of global masked diffusion (Man et al., 2026). MinerU-Diffusion formulates document OCR as inverse rendering and combines a block-wise diffusion decoder with an uncertainty-driven training curriculum (Dong et al., 2026). These models demonstrate that document outputs can be recovered through parallel diffusion decoding and provide favorable accuracy–efficiency trade-offs. Nevertheless, direct diffusion decoding has not consistently matched the recognition accuracy of strong AR OCR systems across all reported benchmarks and operating points, motivating methods that retain parallel proposals without directly committing every diffusion prediction.

AR-Diffusion Hybrid Models. Rather than replacing pretrained AR models with diffusion-only models, recent work adapts AR checkpoints to support both causal and diffusion-style prediction. SDAR studies data-efficient conversion from pretrained AR models to block-wise diffusion models (Cheng et al., 2025). Fast-dVLM extends direct AR-to-diffusion conversion to

vision–language models, jointly training causal and denoising objectives so that the adapted model retains both generation modes (Wu et al., 2026a). Nemotron-Labs-Diffusion scales this formulation to a family of language and vision–language models that can operate in AR, diffusion, or self-speculative modes (Fu et al., 2026). TiDAR more tightly integrates the two modes through a structured attention pattern that combines diffusion-based drafting with AR generation (Liu et al., 2025b).

A related line of work uses diffusion models specifically as drafters for an AR target. Speculative diffusion decoding and DFlash train dedicated diffusion drafters for parallel block proposal (Chen et al., 2026; Christopher et al., 2025). DiffuSpec instead repurposes a pretrained diffusion language model as a training-free drafter, while SpecDiff-2 improves agreement between the diffusion drafter and AR verifier (Li et al., 2025; Sandler et al., 2025). A further line studies how to align a block drafter with left-to-right verification, through position-aware loss weighting, accept-until-fail supervision, or targeted repair of uncertain positions (Liu et al., 2026; Whalen et al., 2026; Wu et al., 2026b; Yang and Li, 2026). Across these approaches, speculative verification allows diffusion prediction errors to reduce accepted draft length rather than directly alter the output of the AR target. Our method follows the shared-model branch of this literature: we adapt the region-level generation model of GLM-OCR to support block-diffusion drafting and causal AR verification within the same parameter set, without introducing a separate draft network.

Reinforcement Learning for Diffusion Language Models. Policy-gradient RL over a diffusion language model requires a sequence likelihood that depends on the denoising trajectory, so existing methods either optimize an approximation of it or derive step-level gradients that remain unbiased (Zhan, 2025; Zhao et al., 2025). Self-speculative decoding has separately been used to accelerate the rollout phase of RL itself (Kim et al., 2026). Our setting avoids the estimation problem rather than solving it: because the adapted model retains an exact causal factorization, GRPO (Shao et al., 2024) can be applied through the AR path, and the shared parameters it updates are the ones the diffusion drafter uses.

6. Conclusion

We presented GRAVITYOCR, a parameter-shared framework that combines block-diffusion drafting with causal AR verification to accelerate document OCR. Joint AR–diffusion training enables a single model to draft token blocks in parallel and verify them before commitment, preserving its AR output while allowing multiple tokens to be committed per round. We further apply GRPO through the causal AR path using sequence- and structure-level OCR rewards, avoiding diffusion-trajectory likelihood estimation while updating the shared drafter and verifier parameters. On OmniDocBench v1.6, this improves the Overall score from 94.92 to 95.16, close to the original GLM-OCR score of 95.48, while preserving diffusion drafting efficiency. In SGLang, GRAVITYOCR achieves a 3.94× decode-only speedup on region crops and a 1.32× end-to-end page-processing speedup over AR decoding. These results show that parameter-shared drafting and verification accelerate document OCR while largely retaining the base model’s recognition capabilities, without a separate drafter network.

References

- M. Arriola, A. Gokaslan, J. T. Chiu, Z. Yang, Z. Qi, J. Han, S. S. Sahoo, and V. Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. International Conference on Learning Representations, 2025.
- J. Austin, D. D. Johnson, J. Ho, D. Tarlow, and R. van den Berg. Structured denoising diffusion models in discrete state-spaces. Advances in Neural Information Processing Systems, 2021.
- L. Blecher, G. Cucurull, T. Scialom, and R. Stojnic. Nougat: Neural optical understanding for academic documents. arXiv preprint arXiv:2308.13418, 2023.
- C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper. Accelerating large language model decoding with speculative sampling. arXiv preprint arXiv:2302.01318, 2023.
- J. Chen, Y. Liang, and Z. Liu. DFlash: Block diffusion for flash speculative decoding. arXiv preprint arXiv:2602.06036, 2026.
- S. Cheng, Y. Bian, D. Liu, L. Zhang, Q. Yao, Z. Tian, W. Wang, Q. Guo, K. Chen, B. Qi, et al. Sdar: A synergistic diffusion-autoregression paradigm for scalable sequence generation. arXiv preprint arXiv:2510.06303, 2025.
- J. K. Christopher, B. R. Bartoldson, T. Ben-Nun, M. Cardei, B. Kailkhura, and F. Fioretto. Speculative diffusion decoding: Accelerating language generation through diffusion. In NAACL, 2025. arXiv:2408.05636.
- C. Cui, T. Sun, S. Liang, T. Gao, Z. Zhang, et al. PaddleOCR-VL: Boosting multilingual document parsing via a 0.9b ultra-compact vision-language model. arXiv preprint arXiv:2510.14528, 2025.
- C. Cui, T. Sun, S. Liang, T. Gao, Z. Zhang, J. Liu, X. Wang, C. Zhou, H. Liu, M. Lin, Y. Zhang, Y. Zhang, Y. Liu, D. Yu, and Y. Ma. Paddleocr-vl-1.5: Towards a multi-task 0.9b VLM for robust in-the-wild document parsing. arXiv preprint arXiv:2601.21957, 2026.
- H. Dong, J. Niu, B. Wang, W. Zeng, W. Zhang, and C. He. Mineru-diffusion: Rethinking document OCR as inverse rendering via diffusion decoding. arXiv preprint arXiv:2603.22458, 2026.
- S. Duan, Y. Xue, W. Wang, Z. Su, H. Liu, et al. GLM-OCR technical report. arXiv preprint arXiv:2603.10910, 2026.
- Y. Fu, L. Whalen, A. Garg, C. Wu, M. Khadkevich, N. Oswald, E. Xie, D. Egert, S. T. Sreenivas, S. Diao, et al. Nemotron-labs-diffusion: A tri-mode language model unifying autoregressive, diffusion, and self-speculation decoding. arXiv preprint arXiv:2607.05722, 2026.
- A. W. Harley, A. Ufkes, and K. G. Derpanis. Evaluation of deep convolutional nets for document image classification and retrieval. In International Conference on Document Analysis and Recognition, 2015.
- M. Kim, M. Lee, S. Oh, K. Galim, D. Kim, C. Hooper, H. Singh, A. Gholami, H. I. Koo, and W. Kang. Efficientrollout: System-aware self-speculative decoding for rl rollouts. arXiv preprint arXiv:2606.18967, 2026.

- W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023. [arXiv:2309.06180](#).
- H. Laurençon, A. Marafioti, V. Sanh, and L. Tronchon. Building and better understanding vision-language models: insights and future directions. [arXiv preprint arXiv:2408.12637](#), 2024.
- Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, 2023.
- G. Li, Z. Fu, M. Fang, Q. Zhao, M. Tang, C. Yuan, and J. Wang. Diffuspec: Unlocking diffusion language models for speculative decoding. [arXiv preprint arXiv:2510.02358](#), 2025.
- G. Li, X. Wan, S. Peng, W. Wang, H. Feng, Y. Du, B. Wu, Z. Ruan, et al. Hunyuanocr-1.5: Making lightweight OCR VLMs faster and better. [arXiv preprint arXiv:2607.04884](#), 2026.
- W. Liao, H. Li, P. Xie, X. Cai, Y. Shen, Y. Xin, Q. Qin, S. Ye, et al. Hsd: Training-free acceleration for document parsing vision-language models with hierarchical speculative decoding. [arXiv preprint arXiv:2602.12957](#), 2026.
- A. Liu, M. He, S. Zeng, S. Zhang, L. Zhang, C. Wu, W. Jia, Y. Liu, X. Zhou, and J. Zhou. WeDLM: Reconciling diffusion language models with standard causal attention for fast inference. [arXiv preprint arXiv:2512.22737](#), 2025a.
- A. Liu, J. Meng, F. Liu, and Y. Chen. Cure: Local uncertainty repair for block-parallel speculative decoding. [arXiv preprint arXiv:2608.00531](#), 2026.
- J. Liu, X. Dong, Z. Ye, R. Mehta, Y. Fu, V. Singh, J. Kautz, C. Zhang, and P. Molchanov. TiDAR: Think in diffusion, talk in autoregression. [arXiv preprint arXiv:2511.08923](#), 2025b.
- S. Lu, Y. Li, Y. Xia, Y. Chen, A.-Y. Ji, et al. Ovisocr2 technical report. [arXiv preprint arXiv:2607.13639](#), 2026.
- S. Man, G. Deutch, R. Ganz, R. Ronen, S. Tsiper, S. Mazor, and N. Nayman. Dodo: Discrete OCR diffusion models. [arXiv preprint arXiv:2602.16872](#), 2026.
- A. Nassar, N. Livathinos, M. Lysak, and P. Staar. Tableformer: Table structure understanding with transformers. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.
- S. Nie, F. Zhu, Z. You, X. Zhang, J. Ou, J. Hu, J. Zhou, Y. Lin, J.-R. Wen, and C. Li. Large language diffusion models. [arXiv preprint arXiv:2502.09992](#), 2025.
- J. Niu, Z. Liu, Z. Gu, B. Wang, L. Ouyang, et al. Mineru2.5: A decoupled vision-language model for efficient high-resolution document parsing. [arXiv preprint arXiv:2509.22186](#), 2025.
- L. Ouyang, Y. Qu, H. Zhou, J. Zhu, R. Zhang, et al. OmniDocBench: Benchmarking diverse PDF document parsing with comprehensive annotations. In *CVPR*, 2025.
- B. Pfitzmann, C. Auer, M. Dolfi, A. S. Nassar, and P. Staar. Doclaynet: A large human-annotated dataset for document-layout segmentation. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022.

- J. Poznanski, A. Rangapur, J. Borchardt, J. Dunkelberger, R. Huff, et al. olmocr: Unlocking trillions of tokens in pdfs with vision language models. arXiv preprint arXiv:2502.18443, 2025.
- rednote hilab. dots.ocr: Multilingual document layout parsing in a single vision-language model. <https://github.com/rednote-hilab/dots.ocr>, 2025.
- S. S. Sahoo, M. Arriola, Y. Schiff, A. Gokaslan, E. Marroquin, J. T. Chiu, A. Rush, and V. Kuleshov. Simple and effective masked diffusion language models. Advances in Neural Information Processing Systems, 2024.
- J. Sandler, J. K. Christopher, T. Hartvigsen, and F. Fioretto. Specdiff-2: Scaling diffusion drafter alignment for faster speculative decoding. arXiv preprint arXiv:2511.00606, 2025.
- Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300, 2024.
- B. Smock, R. Pesala, and R. Abraham. Pubtables-1m: Towards comprehensive table extraction from unstructured documents. In IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022.
- T. Sun, C. Cui, Y. Du, and Y. Liu. Pp-doclayout: A unified document layout detection model to accelerate large-scale data construction. arXiv preprint arXiv:2503.17213, 2025.
- B. Wang, Z. Gu, G. Liang, C. Xu, B. Zhang, B. Shi, and C. He. Unimernet: A universal network for real-world mathematical expression recognition. arXiv preprint arXiv:2404.15254, 2024a.
- B. Wang, F. Wu, L. Ouyang, Z. Gu, R. Zhang, R. Xia, B. Zhang, and C. He. Image over text: Transforming formula recognition evaluation with character detection matching. arXiv preprint arXiv:2409.03643, 2024b.
- B. Wang, T. He, L. Ouyang, F. Wu, Z. Zhao, et al. Mineru2.5-pro: Pushing the limits of data-centric document parsing at scale. arXiv preprint arXiv:2604.04771, 2026.
- H. Wei, Y. Sun, and Y. Li. DeepSeek-OCR: Contexts optical compression. arXiv preprint arXiv:2510.18234, 2025.
- H. Wei, Y. Sun, and Y. Li. DeepSeek-OCR 2: Visual causal flow. arXiv preprint arXiv:2601.20552, 2026.
- L. Whalen, Y. Ito, and R. Sakamoto. Teaching diffusion to speculate left-to-right. arXiv preprint arXiv:2606.11552, 2026.
- T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush. Transformers: State-of-the-art natural language processing. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pages 38–45. Association for Computational Linguistics, 2020.
- C. Wu, H. Zhang, S. Xue, S. Diao, Y. Fu, Z. Liu, P. Molchanov, P. Luo, S. Han, and E. Xie. Fast-dLLM v2: Efficient Block-Diffusion LLM. arXiv preprint arXiv:2509.26328, 2025a.
- C. Wu, H. Zhang, S. Xue, Z. Liu, S. Diao, L. Zhu, P. Luo, S. Han, and E. Xie. Fast-dLLM: Training-free acceleration of diffusion LLM by enabling KV cache and parallel decoding. arXiv preprint arXiv:2505.22618, 2025b.

- C. Wu, S. Lan, Y. Fu, S. Gao, J. Wang, J. Yu, J. M. Alvarez, P. Molchanov, P. Luo, S. Han, L. Zhu, and E. Xie. Fast-dvln: Efficient Block-Diffusion VLM via direct conversion from autoregressive VLM. [arXiv preprint arXiv:2604.06832](#), 2026a.
- T. Wu, Y. Yao, Z. Qi, H. Zheng, Z. Wang, H. Ma, L. Liao, H. Lakkaraju, J. Li, and Y. Du. D-pace: Dynamic position-aware cross-entropy for parallel speculative drafting. [arXiv preprint arXiv:2605.18810](#), 2026b.
- R. Xia, S. Mao, X. Yan, H. Zhou, B. Zhang, H. Peng, J. Pi, D. Fu, W. Wu, H. Ye, S. Wang, J. Ye, B. Wang, A. Zhou, Z. Chen, Q. Zhang, F. Wang, C. Chen, L. Bai, H. Li, J. Yan, and Y. Qiao. Docgenome: An open large-scale scientific document benchmark for training and testing multi-modal large language models. [arXiv preprint arXiv:2406.11633](#), 2024.
- T. Yang and M. Li. Spec-auf: Accept-until-fail training under train-inference misalignment for masked block drafters. [arXiv preprint arXiv:2607.01893](#), 2026.
- J. Ye, Z. Xie, L. Zheng, J. Gao, Z. Wu, X. Jiang, Z. Li, and L. Kong. Dream 7b: Diffusion large language models. [arXiv preprint arXiv:2508.15487](#), 2025.
- A. Zhan. Simple policy gradients for reasoning with diffusion language models. [arXiv preprint arXiv:2510.04019](#), 2025.
- S. Zhao, D. Gupta, Q. Zheng, and A. Grover. d1: Scaling reasoning in diffusion large language models via reinforcement learning. [arXiv preprint arXiv:2504.12216](#), 2025.
- L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, et al. SGLang: Efficient execution of structured language model programs. In [Advances in Neural Information Processing Systems](#), 2024. [arXiv:2312.07104](#).
- X. Zheng, D. Burdick, L. Popa, X. Zhong, and N. X. R. Wang. Global table extractor (gte): A framework for joint table identification and cell structure recognition using visual context. In [IEEE/CVF Winter Conference on Applications of Computer Vision](#), 2021.
- X. Zhong, E. ShafieiBavani, and A. Jimeno Yepes. Image-based table recognition: Data, model, and evaluation. In [European Conference on Computer Vision](#), 2020.

Table 7 | **Training configuration.**

Hardware	16×H100 (2 nodes)
Steps	40,000 (warmup 500, linear decay to 0)
Optimizer	AdamW ($\beta_1=0.9$, $\beta_2=0.999$), no weight decay
Peak LR	2×10^{-5} decoder, 2×10^{-6} vision encoder
Precision / parallelism	bf16, ZeRO stage 2, gradient clipping 1.0
Batch	1 packed row/GPU $\times$ 16 GPUs $\times$ accumulation 4
Packing budget	10,240 forward tokens/row (response cap 3,072)
Block size / mask schedule	$B=32$, $t \sim \mathcal{U}(0, 1)$
Loss weights	$0.5 \mathcal{L}_{\text{diff}} + 0.5 \mathcal{L}_{\text{AR}}$

A. Training Details

Table 7 lists the training configuration. All parameters are trained, including the vision encoder, which uses $0.1\times$ the decoder learning rate; the only architectural addition is the [M] embedding, a new vocabulary row initialized by mean-resizing, i.e. drawn from a normal distribution fitted to the existing embedding statistics. The joint loss is normalized so that its coefficients sum to one, $0.5 \mathcal{L}_{\text{diff}} + 0.5 \mathcal{L}_{\text{AR}}$ for $\lambda = 1$, leaving learning-rate semantics unchanged.

Shared-prefix packing. Because the three response streams of Figure 3 share one image and prompt, a training row costs prompt + vision + $3 \cdot$ response forward tokens, and the packer fills each row against this cost rather than the raw sequence length—the same consideration behind Fast-dVLM’s vision-efficient concatenation (Wu et al., 2026a) and Nemotron’s asymmetric dual stream (Fu et al., 2026). Rows are filled first-fit with carry-over: a sample that does not fit starts the next batch, and typical rows pack 21–30 document samples.

EOS block fill. Each response is padded with exactly B EOS tokens during training, so the model learns to fill the tail block with an EOS run instead of leaking length information through block alignment. The AR loss counts only the first EOS of each run, while the diffusion streams supervise all of them; without this restriction the AR path over-predicts EOS and Overall drops by more than four points.

B. Decoding Details and Equality Testing

B.1. Direct Block-Diffusion Decoding

The completed prefix is represented by causal KV states, and generation proceeds block by block. Each block is initialized with B [M] tokens; under the shifted convention, the prediction for the block’s first position comes from the final position of the preceding block.

At each denoising step, one forward pass predicts all remaining masked positions in parallel, reading the distribution for position p from the logit at position $p - 1$. Every prediction whose token probability exceeds the confidence threshold τ_c is revealed; if none exceeds it, the single most confident position is revealed instead. Revealed tokens remain fixed in later steps.

Once the block is complete, one additional forward pass writes its KV states into the cache. This pass extends the cache only and does not verify or replace any token. Generation stops at the block containing the EOS token, and the output is truncated at the first EOS.

B.2. Self-Speculative Decoding and KV-Cache Update

One self-speculative round of the serving implementation proceeds as follows; Figure 4 illustrates the same round.

1. **State.** The KV cache holds causal states for every committed token except the boundary token x_0 , the last committed token, which was produced as the verifier’s prediction in the preceding round and has not yet been fed to the model as input (the initial boundary is the first token generated during prefill).
2. **Draft forward.** A window is formed from x_0 followed by B [M] tokens. The boundary position attends causally to the cache; the masked positions attend causally to the cache and bidirectionally within the window. One forward pass yields, under the shifted alignment, a_0 from the boundary logit, which is identical to the causal AR prediction, and the draft $d_{1:B}$ from the mask logits. No confidence threshold is applied: the one-shot draft proposes all positions unconditionally.
3. **Verify forward.** The sequence $a_0, d_{1:B}$ is processed under token-level causal attention in a single forward pass, attending to the cache and to x_0 , whose causal KV state was written by the draft forward, and producing the AR predictions $a_{1:B+1}$. The verification pass reuses the draft’s KV slots for the mask positions and overwrites their bidirectional states with causal ones, so the cache that later rounds read is always causal.
4. **Accept.** $A \leq B$ is the length of the longest prefix with $d_j = a_j$. The round commits $a_0, d_{1:A}$, and a_{A+1} ; the committed tokens are the verifier’s own predictions at every position.
5. **Cache update.** The causal KV states of x_0, a_0 and $d_{1:A}$ are retained; the states of the rejected suffix are freed. No bidirectional draft state survives the round.
6. **Next round.** a_{A+1} becomes the next boundary token and obtains its KV state in the next draft forward. If $A = B$, the draft was fully accepted and a_{A+1} is the bonus prediction following the entire block. If any committed token completes a stop condition, the committed run is truncated at that token and the remainder of the round is discarded.

A round therefore commits between two tokens (a_0 and a_1) and $B+2$ tokens (a_0 , the full draft, and the bonus token).

B.3. AR-Equivalence and Equality Testing

Argument. Under top-1 decoding, every committed token is the verifier’s argmax prediction given the committed prefix: a_0 is computed under strictly causal attention at the boundary, each accepted draft token satisfies $d_j = a_j$ by the acceptance rule, and a_{A+1} is the verifier’s own prediction. By induction over rounds, the committed sequence equals the sequence that standalone greedy AR decoding produces from the same checkpoint, in exact arithmetic.

Empirical check. We compare the decoded output strings of the two decoders under identical checkpoint, prompts, region crops, and greedy decoding: the final GRPO checkpoint, both decoders served with SGLang, block size $B=32$, repetition penalty 1.0, and no output cap. On the full English OmniDocBench crop set (8,922 crops), the two outputs are identical on 96.6% of crops. The remaining differences are consistent with near-tie argmax flips under bf16 kernels: the AR and verification forwards use different attention kernels, and at the first divergent position the median top-two logit margin is 0.14 nats. Re-evaluating the 305 first divergences in fp32, the fp32 argmax agrees with the AR token in 228 cases and with the self-speculative token in 77, and 65 of these positions are ties with a margin below 0.05 nats.

C. Speed-Measurement Protocol and Profiles

What each speed table measures. All speed tables and Figure 6 use one H100, full resolution, no output cap (natural EOS), and are run solo (no co-tenant job on the node).

Speed columns of Table 1 — page-level, single stream. The unit is one OmniDocBench page and pages are submitted **one at a time** ($C=1$). The wall clock covers the **entire pipeline including layout analysis** (112–146 ms/page for the GLM-family systems, measured live inside the wall) plus region cropping, recognition and markdown assembly. Systems differ in what happens inside one page, and we keep each system’s own design: region-pipeline models (ours, GLM-OCR) detect ~18 regions and fire those recognition requests concurrently, MinerU runs its own two-step pipeline with a much heavier layout stage (~1.3 s/page), while full-page models (HunyuanOCR, dots.ocr, DeepSeek-OCR-2) answer the page in a single request and have no layout stage. Each competitor runs on its own official inference stack: HunyuanOCR-1.5 on vLLM (Kwon et al., 2023) with the vendor recipe, MinerU2.5 and MinerU2.5-Pro through their native pipeline over a vLLM engine, dots.ocr on a vLLM server, PaddleOCR-VL-1.5 on the PaddleX generative-AI server with a vLLM backend, DeepSeek-OCR-2 and MinerU-Diffusion through their native Transformers and diffusion pipelines, and the GLM-OCR MTP mode on vLLM; the GLM-OCR AR baseline and our model are served with SGLang.

Concurrency regimes. Same client, pages and layout-included boundary as Table 1, measured before RL; the **only** change is how the ~18 region requests of a page are issued: strictly sequentially ($C=1$: self-spec 1.54× over AR, 0.420 vs 0.273 pages/s) or all at once ($C=18$: 1.34×, 0.781 vs 0.581). This isolates how much of the speculative advantage comes from an otherwise idle GPU.

Table 3 — crop-level, batch 1, no layout. The unit is a single ground-truth region crop from the English OmniDocBench set (8,922 crops), fed directly to the model, so **layout is not involved at all** and there is no page assembly. Here end-to-end is the full request wall (vision encoding + prefill + decode); decode-only counts only the time spent generating output tokens, excluding the per-crop vision encoding and prompt prefill.

Figure 6 — crop-level throughput with varying batch size. Throughput is measured before RL on layout-detected crops from 755 English OmniDocBench pages. Layout is run once beforehand and excluded from timing. We measure the time to process all crops, including vision encoding, prompt prefill, and decoding, with a full client queue ($C = \max(64, 4 \times \text{bs})$) and the server’s running-request cap set to the plotted batch size. Table 3 instead uses crops defined by the dataset’s annotated region boundaries, submitted one at a time, and the final GRPO checkpoint.

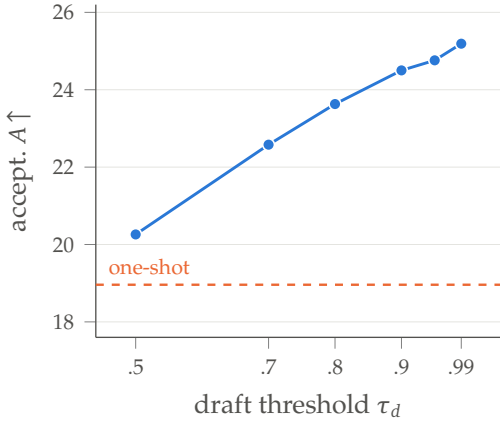
Forward-count accounting. For GLM-OCR MTP and DFlash, TPF counts base-model verification forwards and excludes auxiliary drafting computation. For GRAVITYOCR self-speculation, it counts both full-model draft and verification forwards, excluding prefill. For direct block-diffusion decoding, it counts only forwards that commit at least one token, excluding prefill and the cache-write forward performed after each completed 32-token block. This direct-diffusion convention matches the step-count accounting used for MinerU-Diffusion.

D. Additional Results

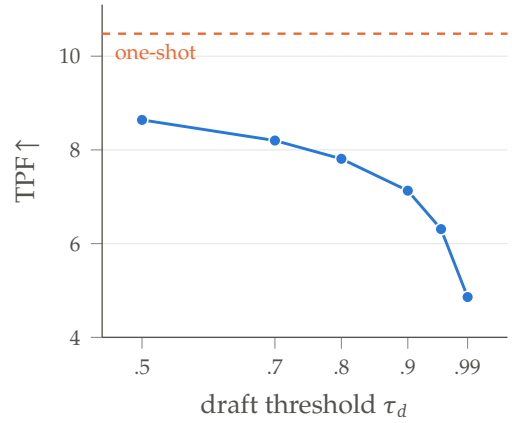
Speedup by Output Length. Table 8 breaks the crop-level comparison of Table 3 down by output length. Both acceptance and speedup rise monotonically with length, from 1.43× below 128 output tokens to 3.68× above 1,024. Two effects compound. Longer outputs spend more of their wall-clock time in decoding, which is the only stage speculation accelerates, so the

Table 8 | **Parallelism and speed by output length.** AR and self-speculative decoding results across output-length ranges. n is the number of crops in each range. Throughput includes vision encoding, prompt prefill, and decoding; speedup is relative to AR.

Output length	n	accepted tokens [†]	TPF [†]	AR tok/s	self-spec tok/s	speedup
< 128	7,257	14.8	8.4	357	510	1.43×
128–511	1,496	18.1	10.1	583	1,176	2.02×
512–1023	92	18.4	10.2	737	2,033	2.76×
≥ 1024	77	23.9	12.9	761	2,805	3.68×



(a) Accepted draft tokens per round



(b) Tokens per forward

Figure 7 | **Multi-step drafting versus one-shot drafting.** Instead of filling the masked block in a single forward pass, the draft is refined over several denoising steps: at each step the positions whose confidence exceeds the threshold τ_d are fixed and the rest are re-drafted, and the finished draft is then verified by the causal AR pass exactly as before, so the committed output is identical at every point. (a) A higher threshold spends more steps on the draft and more of its tokens are accepted per round. (b) Every extra step is another forward pass, and the tokens it adds do not pay for it: tokens per forward falls monotonically, and one-shot drafting (dashed) commits the fewest tokens per round yet delivers the highest tokens per forward.

fixed per-crop vision and prefill cost stops diluting the gain. At the same time, acceptance itself grows with length, from 14.8 of 32 drafted tokens per round below 128 output tokens to 23.9 above 1,024—long regions are dominated by structured, syntax-constrained spans that the one-shot draft predicts well—so each verification also commits more tokens. Four fifths of OmniDocBench crops fall in the shortest bin, which is why the crop-averaged speedup of 1.74× sits well below the longest bin; the content-type breakdown in Table 4 is largely this length effect in disguise, with table regions both the longest and the fastest-accelerating outputs.

Draft Schedule. Figure 7 sweeps the number of denoising steps used to build the draft, controlled by the draft-side confidence threshold τ_d , measured on 400 English OmniDocBench crops. Refining the draft over more steps raises the accepted length per round (Figure 7(a)) but costs additional draft forwards, and the extra forwards cost more than the tokens they buy: tokens per forward falls monotonically as the draft is refined (Figure 7(b)). One-shot drafting is therefore the operating point we deploy.

Table 9 | **All-mask training does not produce a better draft.** The all-mask model commits fewer tokens per forward at every checkpoint, and its accuracy is behind at five of six.

tokens per forward [†] (self-speculative)	2k	4k	5k	6k	8k	10k
All-mask	4.04	4.82	5.54	5.94	6.45	6.66
Uniform t (ours)	4.64	5.30	6.04	6.00	6.78	6.75
OmniDocBench Overall [†]	2k	4k	5k	6k	8k	10k
All-mask	94.49	94.59	94.75	94.66	94.76	94.86
Uniform t (ours)	94.71	94.74	94.76	94.55	94.92	95.02

Direct-Diffusion Quality. Table 6 also reports the diagnostic direct-diffusion mode of Appendix B.1 on the official protocol at four confidence thresholds. After GRPO, quality rises monotonically with τ_c while parallelism falls; even the best direct-diffusion operating point trails self-speculative decoding on both axes simultaneously.

Mask Schedule. Table 9 compares the uniform partial-masking objective against an all-mask variant matched in every other training setting, evaluated at the same step counts on the self-speculative path. The all-mask model is behind on quality at five of six checkpoints and commits fewer tokens per forward, indicating that the partial-masking objective, not merely exposure to masked positions, is what makes the draft strong.

The $\tau_c=0.95$ Gap After GRPO. In Table 6, direct diffusion gains +2.91 Overall at $\tau_c=0.95$ while every other threshold moves by less than 0.4. The pre-GRPO model degrades three times as many formulas at exactly that setting—116 versus 37 expressions scoring zero CDM out of 2,352—which is also why its accuracy is non-monotone in τ_c while the post-GRPO curve is not.

E. RL Details

Optimization. We apply GRPO with full fine-tuning of the entire model (no LoRA), starting from the jointly trained 40k checkpoint. Each step draws 24 prompts with 28 rollouts per prompt (672 completions); advantages are standardized within each group, the loss is token-level, and clipping is asymmetric ($\epsilon_{\text{low}}=0.2$, $\epsilon_{\text{high}}=0.28$). A KL penalty with $\beta=10^{-3}$ anchors the policy to a frozen copy of the initial checkpoint. Training uses a learning rate of 3×10^{-6} with cosine decay, 8-bit AdamW, bf16, and gradient clipping 1.0; checkpoints are saved every 250 steps, and we report step 500 (about 1.6 epochs over the prompt pool). Truncated completions are masked out of the loss. No diffusion-specific objective is applied during GRPO; the drafter changes only through the shared parameters updated by the AR-path loss. On the 8,922 English OmniDocBench crops, tokens per forward are 9.61 before RL and 9.68 at step 500.

Rollouts. Rollouts are sampled from the causal AR policy at temperature 1.0 without nucleus truncation, with a maximum completion length of 8,448 tokens, using a colocated vLLM engine (Kwon et al., 2023). Policy log-probabilities are recomputed by the training framework through the same causal factorization, with token-level importance-sampling correction between the rollout and training kernels.

Rewards. All rewards lie in $[0, 1]$; CDM is not used as a reward. Both the prediction and the reference are first normalized with the same markup normalization the official OmniDocBench scorer applies, so the reward cannot be improved by markup conventions the evaluation ignores. Plain text is scored by the normalized Levenshtein similarity of the two normalized strings. For tables, let TEDS_s be the structure-only TEDS and s_{cell} the normalized Levenshtein similarity of the concatenated cell strings in reading order; the reward is

$$\begin{aligned} r_{\text{table}} &= \text{clip}_{[0,1]}((0.45 \text{TEDS}_s + 0.55 s_{\text{cell}}) (1 - p_{\text{row}}) (1 - p_{\text{loop}})), \\ p_{\text{row}} &= 0.35 \min(1, \text{over}/d) + 0.35 \min(1, \text{under}/d), \quad d = \max(4, n_{\text{gt}}), \\ p_{\text{loop}} &= \begin{cases} 0 & f = 0, \\ \min(1, 0.30 + 0.70 f) & f > 0, \end{cases} \end{aligned}$$

where *over* and *under* are the row-count excess and shortfall relative to the n_{gt} reference rows, and f is the fraction of predicted rows that are excess repeats of a row key beyond $\max(3, n_{\text{gt}}(\text{key}) + 1)$, zero if no row repeats that often. The result is then raised toward one by an exact-grid bonus, $r \leftarrow r + b(1 - r)$, where b is 0.05 if the predicted row count matches the reference plus 0.05 if the predicted column count matches. An unclosed `<table>` scores zero (the evaluator drops such tables), and a table that cannot be parsed scores 0.15 times the plain edit similarity. For formulas, the reward is $r_{\text{formula}} = \text{sim}(\text{canon}(\hat{y}), \text{canon}(y)) \cdot 0.3^\nu$, where *sim* is normalized Levenshtein similarity and *canon* strips math delimiters, maps `\dfrac` and `\tfrac` to `\frac`, and removes `\left/\right.`, spacing commands, whitespace, and braces; ν counts the well-formedness checks (`\left/\right.` balance, brace balance, `\begin/\end` environments, and unescaped \$ parity) failed by the prediction but not by the reference. Every reward is finally multiplied by a degeneration guard $g = \min(\rho_8, \max(0, 1 - (L_{\text{run}} - 20)/200))$, where ρ_8 is the distinct 8-gram ratio, computed only for outputs of at least 16 whitespace-separated tokens and taken as one otherwise, and L_{run} is the longest single-character run.

Prompt pool. The pool contains 7,723 region crops from public document datasets of the same kind as the training pool; it contains no benchmark images, and the PubTabNet portion uses its training split only. Its composition is 92% tables, 6% text, and 2% formulas. References are the source datasets’ original annotations for the table and formula crops (94% of the pool) and teacher transcriptions for the text crops. Candidate prompts were screened by sampling eight rollouts each and discarding prompts whose reward variance is zero (their group advantage vanishes under GRPO), and the remainder was mined toward hard and long tables; the median reference length is about 1,900 characters.

F. Qualitative Examples

Figures 8 to 10 show self-speculative decoding with the final GRPO checkpoint on OmniDocBench crops of the three region types. For each crop we show the raw serialized output with every character shaded by the decoding round that committed it; one round is one bidirectional draft forward over the masked block followed by one causal verify forward, so the number of shaded segments is the number of forward pairs that produced the output. A red tick marks a position where the verifier rejected the draft and committed its own token instead. Every round commits a_0 , the accepted draft prefix and the verifier’s own next token, so each round advances by at least two tokens; `<EOS>` marks the end-of-sequence token, shaded with the round that emitted it. Each figure carries a legend of the rounds it contains, and each block is headed by its number of rounds and tokens. For all crops shown, the self-speculative output is byte-identical to autoregressive greedy decoding of the same weights.

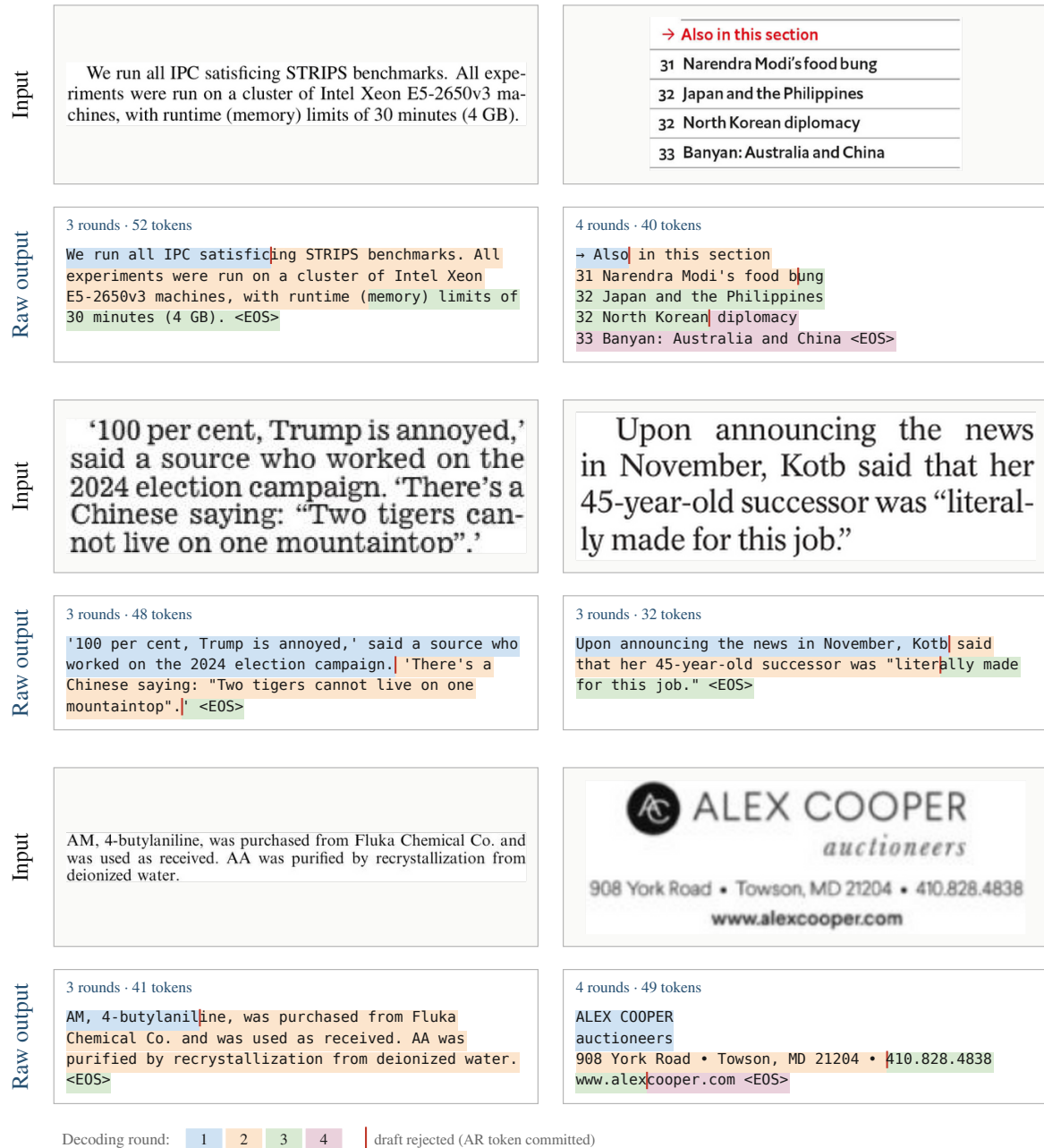


Figure 8 | **Qualitative results on text regions.** Self-speculative decoding on OmniDocBench. Each block shows the input crop (top) and the raw output (bottom); each color denotes a different decoding round, a red tick marks a rejected draft token, and <EOS> is the end-of-sequence token.

Input	$\left[\begin{array}{ccccc c} 3 & -6 & -1 & 1 & 5 & 0 \\ -1 & 2 & 2 & 3 & 3 & 0 \\ 4 & -8 & -3 & -2 & 1 & 0 \end{array} \right]$	$\frac{ x-r_i }{ r_i } = 1-x/r_i \geq 1- x/r_i \geq 1/2 > \varepsilon.$
Raw output	5 rounds · 80 tokens <code>\left[\begin{array}{ccccc c} 3 & -6 & -1 & 1 & 5 & 0 \\ -1 & 2 & 2 & 3 & 3 & 0 \\ 4 & -8 & -3 & -2 & 1 & 0 \end{array} \right] <E0S></code>	5 rounds · 77 tokens <code>\frac{\left x - r_{\textit{i}} \right }{\left r_{\textit{i}} \right } = \left 1 - x / r_{\textit{i}} \right \geq 1 - \left x / r_{\textit{i}} \right \geq 1 / 2 > \varepsilon . <E0S></code>
Rendered	$\left[\begin{array}{ccccc c} 3 & -6 & -1 & 1 & 5 & 0 \\ -1 & 2 & 2 & 3 & 3 & 0 \\ 4 & -8 & -3 & -2 & 1 & 0 \end{array} \right]$	$\frac{ x-r_i }{ r_i } = 1-x/r_i \geq 1- x/r_i \geq 1/2 > \varepsilon.$
Input	$(1+r_1)^{1/n} \cdots (1+r_n)^{1/n} \geq 1 + (r_1 \cdots r_n)^{1/n}.$	$\textcircled{a} d^9 \textcircled{b} 4^{14x} \textcircled{c} 8y^4 \textcircled{d} w^6$
Raw output	4 rounds · 60 tokens <code>(1 + r_{\textit{1}})^{\textit{1} / \textit{n}} \dots (1 + r_{\textit{n}})^{\textit{1} / \textit{n}} \geq 1 + \left(r_{\textit{1}} \dots r_{\textit{n}} \right)^{\textit{1} / \textit{n}} . <E0S></code>	5 rounds · 56 tokens <code>\textcircled{a} d^{\textit{9}} \textcircled{b} 4^{\textit{14} x} \textcircled{c} 8 y^{\textit{4}} \textcircled{d} w^{\textit{6}} <E0S></code>
Rendered	$(1+r_1)^{1/n} \cdots (1+r_n)^{1/n} \geq 1 + (r_1 \cdots r_n)^{1/n}.$	$\textcircled{a} d^9 \textcircled{b} 4^{14x} \textcircled{c} 8y^4 \textcircled{d} w^6$
Input	$[[p \lambda q]_{\mathcal{N}_h \lambda + \mu} \alpha(r)]_{\mathcal{N}_h}$	$\mathcal{O}_N(\vec{x} v) = \langle \emptyset, v \tilde{\mathcal{R}} _{(\mathcal{V}_{q_1}^*)^{\otimes N} \otimes \mathcal{F}_{q_2, q_3}^{(1,0)}(v)} \emptyset, v \rangle =$
Raw output	4 rounds · 41 tokens <code>[[p _ { \lambda } q] _ { \mathcal{N} _ { h } \lambda + \mu } \alpha (r)] _ { \mathcal{N} _ { h } } <E0S></code>	5 rounds · 105 tokens <code>\mathcal{O} _ { N } (\vec{x} v) = \langle \emptyset , v \tilde{\mathcal{R}} _ { (\mathcal{V} _ { q_1 } ^ { * }) ^ { \otimes N } \otimes \mathcal{F} _ { q_2 , q_3 } ^ { (1 , 0) } (v) } \emptyset , v \rangle =</code>
Rendered	$[[p \lambda q]_{\mathcal{N}_h \lambda + \mu} \alpha(r)]_{\mathcal{N}_h}$	$\mathcal{O}_N(\vec{x} v) = \langle \emptyset, v \tilde{\mathcal{R}} _{(\mathcal{V}_{q_1}^*)^{\otimes N} \otimes \mathcal{F}_{q_2, q_3}^{(1,0)}(v)} \emptyset, v \rangle =$

Decoding round: 1 2 3 4 5 | draft rejected (AR token committed)

Figure 9 | **Qualitative results on formula regions.** Self-speculative decoding on OmniDocBench. Each block shows the input crop, the $\text{\LaTeX}$ output and its rendering; each color denotes a different decoding round, a red tick marks a rejected draft token, and $\text{\textless E0S\textgreater}$ is the end-of-sequence token.

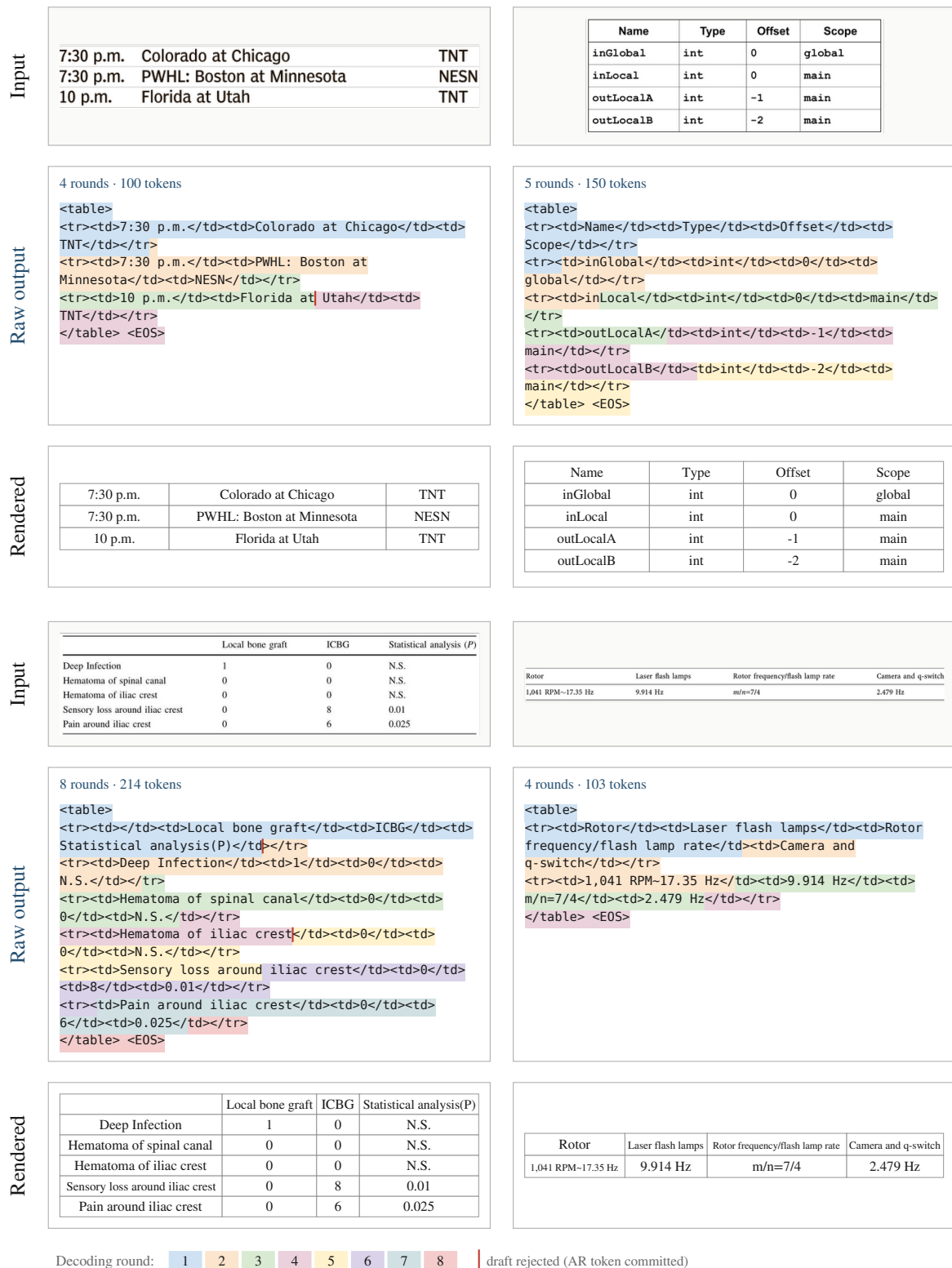


Figure 10 | **Qualitative results on table regions.** Self-speculative decoding on OmniDocBench. Each block shows the input crop, the HTML output and its rendering; each color denotes a different decoding round, a red tick marks a rejected draft token, and <EOS> is the end-of-sequence token.